

Backend Architecture

DDD-lite + Layered | WalkMate Android Project

Overview

Model: DDD-lite + Layered architecture.

Organizational orientation:

- **domain/** — domain-oriented (organized by aggregate root)
- **application/** — feature-oriented (organized by use case)

Goals:

- Separate business logic from technical implementation
- Keep domain independent from framework, DB, and HTTP
- Easy to extend when adding new domains or features

1. Standard Folder Structure

```
src/main/java/com/walkmate/walkmate/  
├─ application/  
│   └─ <domain-name>/  
│       ├── <Domain>CommandService.java  
│       └─ <Domain>QueryService.java  
├─ domain/  
│   ├── <domain-name>/  
│   │   ├── <AggregateRoot>.java  
│   │   ├── <Domain>Repository.java  
│   │   ├── <Domain>ErrorCode.java  
│   │   ├── <ValueObject>.java  
│   │   └─ <EnumOrPolicy>.java  
│   └─ shared/  
│       ├── exception/  
│       │   ├── DomainException.java  
│       │   └─ ErrorCode.java  
│       └─ valueobject/  
├─ infrastructure/  
│   └─ repository/  
│       └─ <domain-name>/
```

```

├── <Domain>JooqRepository.java
├── config/
├── security/
├── exception/
├── presentation/
│   ├── controller/
│   │   ├── <domain-name>/
│   │   │   └── <Domain>Controller.java
│   ├── dto/
│   │   ├── request/
│   │   │   ├── <domain-name>/
│   │   │   │   └── <Verb><Domain>Request.java
│   │   └── response/
│   │       ├── <domain-name>/
│   │       │   ├── <Domain>Response.java
│   │       │   └── <Domain>SummaryResponse.java
│   ├── mapper/
│   └── exception/
└── Application.java

```

2. Layer Responsibilities

Layer	Orientation	Responsibility
application/	Feature-oriented	Coordinates use cases, transaction boundary, calls domain repository interface. No business rules.
domain/	Domain-oriented	Core business rules, entity/value object/policy, repository contract, domain-scoped errors.
infrastructure/	Technical	DB/query/framework/security implementation. Repository implements domain interface via jOOQ. Only technical exceptions here.
presentation/	HTTP entry point	Controller, DTO mapping, validation, exception-to-response conversion.

Note on domain/shared/: only use for value objects and exceptions reused by multiple domains. Do not use as a junk drawer.

3. Naming Conventions

The domain name drives naming across all layers. If the domain is intent, every layer uses intent as the folder and Intent as the class prefix.

3.1 Class naming by layer

Layer	Pattern	Example
domain/ — aggregate	<Domain>.java	Intent.java
domain/ — repo interface	<Domain>Repository.java	IntentRepository.java
domain/ — error codes	<Domain>ErrorCode.java	IntentErrorCode.java
application/ — writes	<Domain>CommandService.java	IntentCommandService.java
application/ — reads	<Domain>QueryService.java	IntentQueryService.java
infrastructure/ — repo impl	<Domain>JooqRepository.java	IntentJooqRepository.java
presentation/ — controller	<Domain>Controller.java	IntentController.java
presentation/ — request DTO	<Verb><Domain>Request.java	CreateIntentRequest.java
presentation/ — response DTO	<Domain>Response.java	IntentResponse.java
presentation/ — list DTO	<Domain>SummaryResponse.java	IntentSummaryResponse.java

3.2 Request DTO — use the verb

Generic names like IntentRequest do not communicate which operation is being performed. Use the verb as a prefix.

Avoid	Use instead
IntentRequest	CreateIntentRequest
SessionRequest	StartSessionRequest / EndSessionRequest
RatingRequest	SubmitRatingRequest

3.3 Response DTO — single vs list

Distinguish between full and lightweight response shapes to prevent over-fetching in list endpoints.

```
// Single object — full detail
IntentResponse.java
```

```
// Lightweight — for match cards / list views
IntentSummaryResponse.java
```

3.4 Repository — impl suffix carries the technology

Never use the generic Impl suffix. The technology name (Jooq) tells readers exactly what they are looking at.

Avoid	Use instead
IntentRepositoryImpl	IntentJooqRepository

3.5 Service method naming — respect the CQRS split

Methods in CommandService must write. Methods in QueryService must read. If you find yourself crossing the boundary, the method belongs in the other class.

```
// IntentCommandService.java
createIntent(...)
cancelIntent(...)

// IntentQueryService.java
findNearbyMatches(...)
getIntentById(...)
```

3.6 Error codes — domain prefix always

Prefix every error code constant with the domain name in UPPER_SNAKE_CASE. Without the prefix, NOT_FOUND and EXPIRED become ambiguous across 4+ domains.

```
// IntentErrorCode.java
INTENT_NOT_FOUND
INTENT_ALREADY_CONFIRMED
INTENT_EXPIRED

// SessionErrorCode.java
SESSION_NOT_STARTED
SESSION_ALREADY_ENDED
```

4. Standard Request Flow

```
Controller
→ <Domain>CommandService / <Domain>QueryService
→ Domain Model
→ <Domain>Repository (interface)
→ <Domain>JooqRepository (infrastructure impl)
→ Database
```

5. Core Principles

- Domain is the center. Infrastructure and presentation depend on domain, never the reverse.
- Organize by feature first, split by type only when genuinely needed.
- DTO stays in presentation only. Never let it leak into domain or application layers.
- Repository interface lives in domain. Implementation lives in infrastructure/repository/<domain-name>/ using jOOQ.
- domain/shared/ is for reused cross-domain objects only — not a catch-all folder.
- infrastructure/exception/ holds only technical exceptions (DB, external services). Business exceptions live in domain/<domain-name>/<Domain>ErrorCode.java.

6. Quick Reference Cheat Sheet

Token	Meaning	Example
<domain>	Lowercase domain folder name	intent, session, user, rating
<Domain>	PascalCase class prefix	Intent, Session, User, Rating
<Domain>Repository	Interface in domain/	IntentRepository
<Domain>JooqRepository	Impl in infrastructure/	IntentJooqRepository
<Domain>CommandService	Writes in application/	IntentCommandService
<Domain>QueryService	Reads in application/	IntentQueryService
<Verb><Domain>Request	Request DTO	CreateIntentRequest
<Domain>Response	Single result DTO	IntentResponse
<Domain>SummaryResponse	Lightweight list DTO	IntentSummaryResponse
<DOMAIN>_<STATE>	Error code constant	INTENT_NOT_FOUND